USB 3.2 Composite Device (Bandwidth Test + DFU) System Architecture

Document Version: V1.1 | **Date:** 2026-11-04
Project Description: FPGA-based USB 3.2 Bandwidth Test Device (with Background DFU)

Revision History

Version	Date	Description	Author
V1.0	2026-10-14	Initial release. Implemented USB 3.2 Bandwidth test function, supporting reversible plug.	zefeng
V1.1	2026-11-04	Added driverless DFU functionality.	zefeng

1. System Overview

This project implements an FPGA-based USB 3.2 composite device, featuring two core functions: **USB Read/Write Bandwidth Test (RWBW)** and **Background Device Firmware Upgrade (DFU)**. The RWBW test works with the accompanying software `USB_RWBWTest_V1.0.1` to perform read/write consistency testing and stress testing.

The system supports background online upgrades based on the standard DFU protocol. To ensure high reliability and prevent the device from being "bricked", the underlying architecture uses a Multi-Boot mechanism that strictly partitions the 8MB (GD25Q64ESIG) or 16MB (W25Q128JVSIQ) SPI Flash space:

- **Working Image**: Runs at the base address `0x0000000`, serving as the target area for DFU updates.
- **Golden Image**: Burned at `0x0300000`, acting as the underlying security baseline for system recovery.
- **User Storage Area**: The remaining space (`0x0600000` and above) is allocated to the UMS function for data storage.

> 💡 **Address Configuration Note:**
> The base addresses mentioned here and below (e.g., `0x0300000`) are referenced for `.bin` bitstream files generated for the **GW5AT-60** series. Users should adjust these addresses based on the specific chip model used in actual development. Reference examples:
>
> - **GW5A-60**: The `.bin` file takes about 800 KB. It is recommended to adjust the golden image address to `0x0100000`.
> - **GW5A-138**: The `.bin` file takes about 4850 KB. It is recommended to adjust the golden image address to `0x0500000`.

The top-level module `Top` integrates the clock network, USB physical layer (SoftPHY), USB protocol layer, and specific business subsystems. This architecture enables data interconnection from the USB interface to the underlying SPI Flash and physical logic pins.

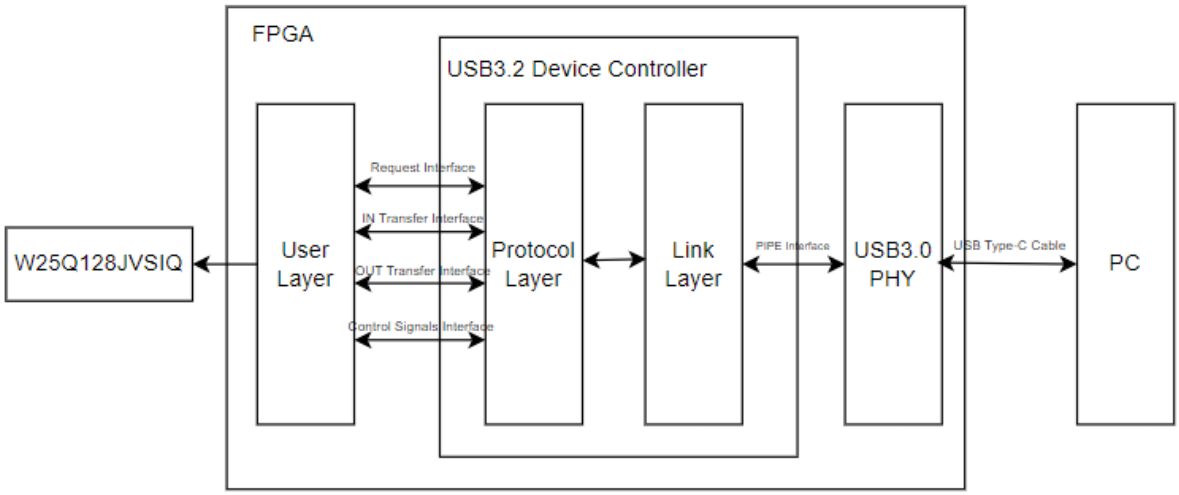
2. External Hardware Interfaces

The physical pin matrix exposed by the top-level `Top` module is as follows:

Category	Signal/Pin Name	Direction	Description
System Control	`CLK_IN`	Input	50MHz base system clock input
	`LED`	Output	Reset completion and system status indicator
USB Physical Layer	`\src\ip\serdes\serdes.csr`	Inout	USB 3.2 related interfaces
SPI Flash	`spi_sclk`	Output	SPI clock
	`spi_cs`	Output	SPI chip select (active low)
	`spi_mosi`	Output	SPI Master Out / Slave In (MOSI) data line
	`spi_miso`	Input	SPI Master In / Slave Out (MISO) data line

3. Core Subsystem Breakdown

The system is logically partitioned into the following main subsystems, which collaborate to achieve USB 3.2 high-speed data stream processing and online firmware upgrades.



3.1 Clock & Reset Management

- **Clock Network**: The system receives external differential clocks via `TLVDS_IBUF` and outputs a 200MHz reference clock via `ELVDS_OBUF` to drive the physical layer. The core control logic working clock (`pclk`) is provided directly by the USB 3.2 PHY.
- **Multi-level Reset Logic**: The top-level logic features built-in boot delay counters to implement a multi-stage asynchronous reset and safe synchronous release sequence (`sys_rst_n` , `phy_resetn` , `phy_resetn2`). This prevents metastability during initial power-up and ensures safe initialization of the SerDes PLL.

3.2 USB Core Framework

- `SerDes_Top` **(USB 3.2 Physical Layer)**: Utilizes the FPGA's underlying high-speed hard SerDes resources to implement USB 3.2 physical signal transmission and reception, providing standard PIPE data buses (e.g., `phy_pipe_rx_data` , `phy_pipe_tx_data`) and control interfaces to the protocol layer.
- `USB3_2_Device_Controller_Top` **(USB 3.2 Core Controller)**: Handles the link and protocol layers for high-speed USB communication. Responsible for maintaining the LTSSM (Link Training and Status State Machine), packet assembly/parsing, CRC, data scrambling, and routing bus transactions to standardized read/write endpoint requests.

3.3 User Layer

The top-level `UserLayer_top` module coordinates various control requests and specific business data interactions:

- **Control Transfer & Enumeration**: Parses Setup control requests sent by the host to EP0 and returns device descriptors to complete enumeration and parameter configuration.
- **Bandwidth Test Module**: Interfaces with the core controller's IN/OUT memory interfaces. It performs lossless read/write loopback on Endpoint 1 (EP1) and generates a continuous high-speed

incremental data stream on Endpoint 2 (EP2) for host-side bandwidth limits and stress testing.

3.4 DFU (Device Firmware Upgrade) Background Mechanism

The driverless online upgrade functionality is deeply integrated into the user logic and top-level state machine:

- **Mode Switch & Soft Detach**: When the host issues a DFU_DETACH request, the control module in `UserLayer_top` triggers the `o_dfu_detach_flag`. Upon capturing this pulse, the top-level logic flips the current operation mode (`dfu_mode`) and starts a 500ms forced timer (`disconnect_timer`).
- **Bus Re-enumeration**: During the 500ms timer, the system pulls down the `usb_logic_rstn` signal (simulating a VBUS power loss), forcing the PC to recognize a physical device disconnection. Once the timer expires, the device resumes operation and re-enumerates under the new mode.
- **SPI Flash Write Routing**: After successful DFU enumeration, firmware data packets received via the endpoint are converted into physical SPI timing and routed to overwrite the `0x0000000` Working Image area in the external SPI Flash, without interfering with data in other sectors.

4. USB_RWBWTest Software User Guide

USB_RWBWTest_V1.0.1 is a USB device read/write and bandwidth testing tool supporting both Simplified Chinese and English interfaces. It enables high-intensity read/write consistency verification and maximum throughput testing.

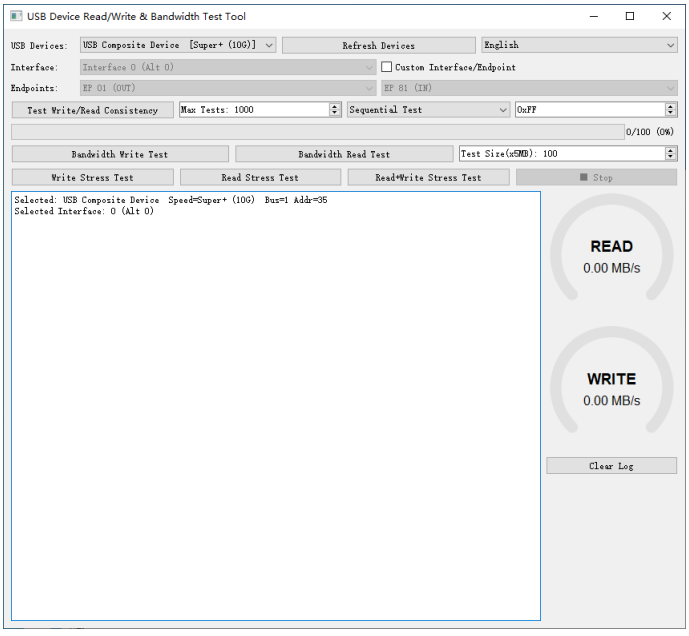


Figure 4-1: USB_RWBWTest_V1.0.1 Main Interface

4.1 Device Connection and Basic Configuration

- **Auto-Enumeration & Refresh:** The software features a background USB hardware topology monitoring thread for plug-and-play and auto-disconnection detection. Users can also click the

"Refresh" button to reload manually.

- **Endpoint Configuration:** By default, the software automatically selects Interface 0 and its corresponding read/write endpoints (Endpoint 1 for consistency testing, Endpoint 2 for stress testing).
- To test specific endpoints, check the "Custom Interface/Endpoint" box to manually assign IN and OUT endpoints.

4.2 Read/Write Consistency Test (Loopback)

The consistency test verifies the accuracy of the FPGA endpoint's data transmission and reception by comparing the sent payload against the returned payload.

- **Test Modes:** Provides three data generation strategies: **Sequential Data Test**, **Fixed Value Test** (customizable base Hex value), and **Random Data Test**.
- **Data Sampling:** When the test begins, a 512-byte sample of the first successfully transferred data (Hex preview of OUT and IN data) is printed in the left log window, while the progress bar displays the current status and success count.

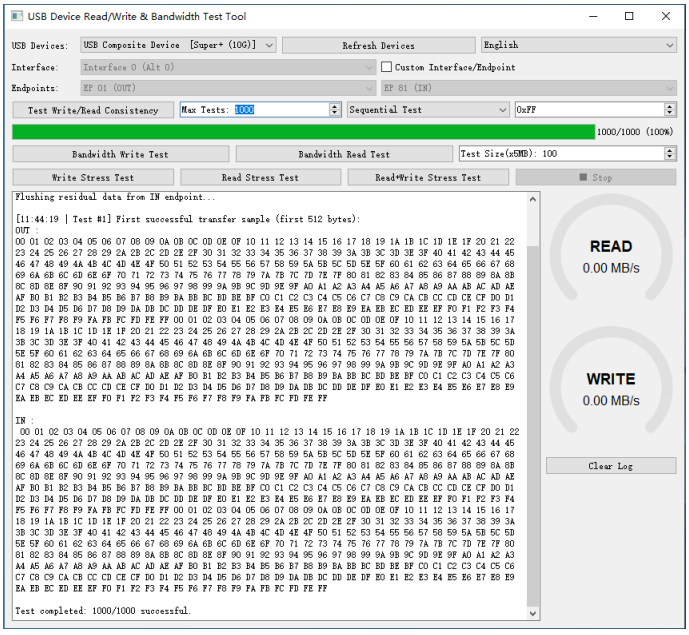


Figure 4-2: Read/Write Consistency Test Status

4.3 Bandwidth & Stress Test

These tests evaluate the actual transmission speed in the current environment. Real-time speed statistics are provided and fed back into the dynamic READ/WRITE dials on the right side of the interface.

- **Bandwidth Test:** Supports standalone Write Bandwidth and Read Bandwidth tests. Users can adjust the loop count per test using the "Test Size (x5MB)" spin box (default is 100 loops, equivalent to 500MB).

- **Stress Test:** Offers Write Stress, Read Stress, and Dual-thread Read+Write Stress tests. Once started, the software performs continuous data throughput with no upper limit until the user manually clicks the "Stop" button.

> ⚠ **Timeout Warning:**
> If the IN endpoint receives no data during the test, the log window will highlight a timeout warning. If this occurs, please verify that the selected endpoint supports bandwidth testing.

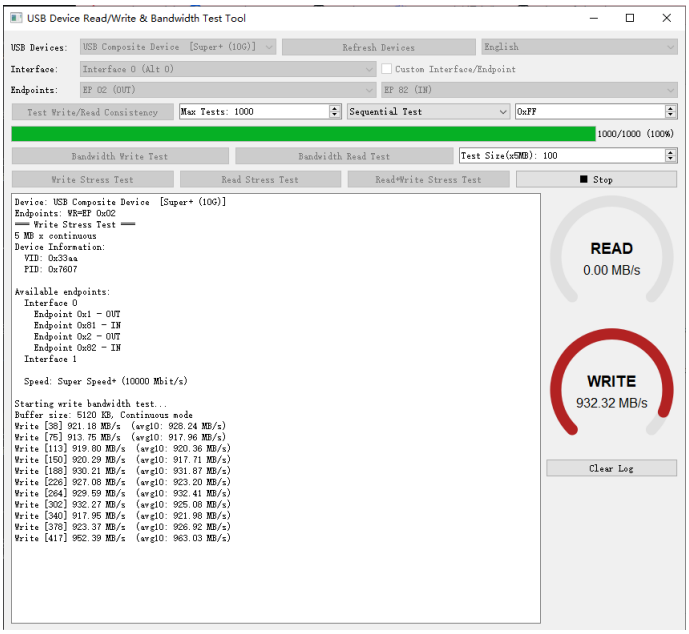


Figure 4-3: Dual-thread Bandwidth Stress Test with Dashboard

5. Functional Testing - Background DFU

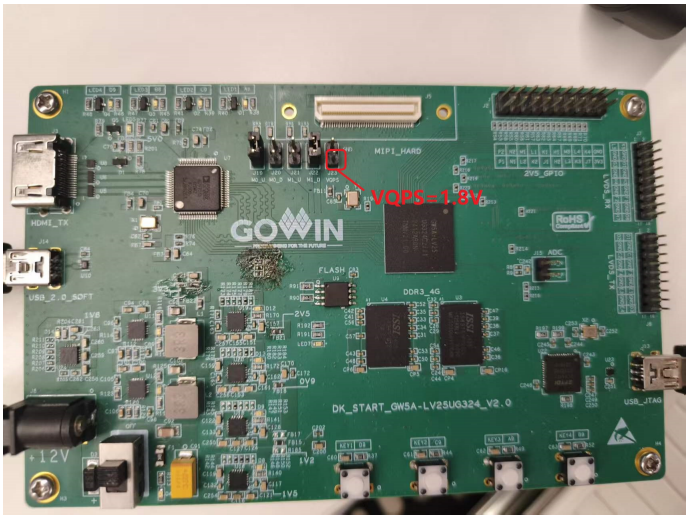
This system uses a Multi-Boot architecture. For secure remote upgrades and data isolation, the physical addresses of the external W25Q128JVSIQ SPI Flash (16MB) are mapped as follows:

- `0x000000`: Working Image, the default download and erase target address for the DFU protocol.
- `0x300000`: Golden Image, a factory-burned secure backup area. This address is customizable.

5.1 FPGA Chip Configuration Parameters Preset

> ⚠ **Development Board Warning:**
> On the `DK_USB_GW5AT-LV60UG225_V1.0/V1.1` board, the **VQPS pin is pulled to GND by default**, meaning the Golden Address write operation cannot be performed. In this case, please skip directly to `5.4 Using dfu-util for Upgrade Testing`.

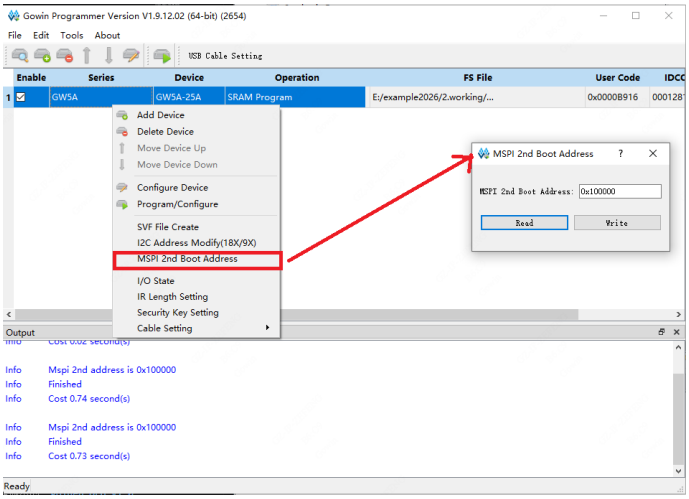
> ⚠ **Hardware Operation Warning:**
> The **Golden Boot Address** and **AES Decryption Key** settings below rely on the underlying eFuse mechanism, which has an **irreversible one-way write limitation** (once a bit is flipped from `0` to `1`, it can never be restored). Please verify carefully before proceeding!
> Additionally, prior to writing these parameters, you **must provide a stable 1.8V supply to the VQPS pin** of the development board; otherwise, the soft configuration write will fail.



		MIN	MAX
V_{EFUSE} [2][3]	Voltage required for eFuse writing	1.62V	1.98V

Setting the Golden Boot Address:

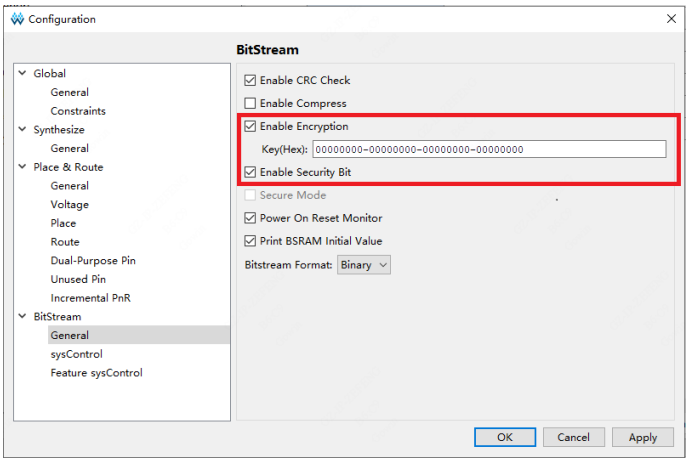
1. Open Gowin Programmer, select the current device, right-click, and select the **"MSPI 2nd Boot Address"** option to enter the configuration page.
2. Click "Read" to fetch the current boot address. It may display `0x800_000`, but the default is actually `0x000_000`.
3. Enter the target boot address in the input box (for this project, change it to `0x300_000`) and click the "Write" button. If it prompts "Write Success", the operation is successful.
4. Click "Read" again to confirm the displayed address matches your input, ensuring the configuration has taken effect.



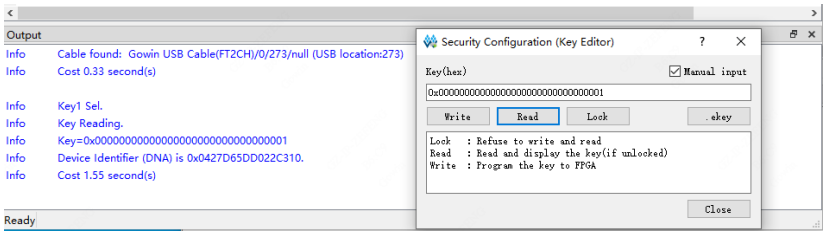
Setting the AES Key:

The Arora V family of FPGA products supports bitstream encryption using the 128-bit AES algorithm. Users can embed the AES key during compilation. Upon power-up, the device will automatically identify encrypted bitstreams matching the preset key or unencrypted bitstreams, while skipping mismatched encrypted bitstreams.

1. **Input Encryption Key**: In the menu bar, select "Project - Configuration", click "BitStream", check "Enable Encryption", and enter the key value as shown below. During subsequent compilations, Gowin EDA will automatically embed this key into the bitstream file.



2. **Set Decryption Key**: Open Gowin Programmer, select the current device, and right-click to choose "Security Key Setting". Enter the encrypted key value in the pop-up interface and click "Write" to burn it into the FPGA, as shown below:



For more detailed instructions, please refer to the "UG714-Arora V 25K FPGA Products Programming and Configuration Guide" —— 4.3 Security

5.2 Golden Image and Working Image Preset

To ensure the anti-brick mechanism functions correctly, a stable version of the `golden_DFU_V1.0.fs` firmware must be burned into the Golden Address before delivery or initial testing.

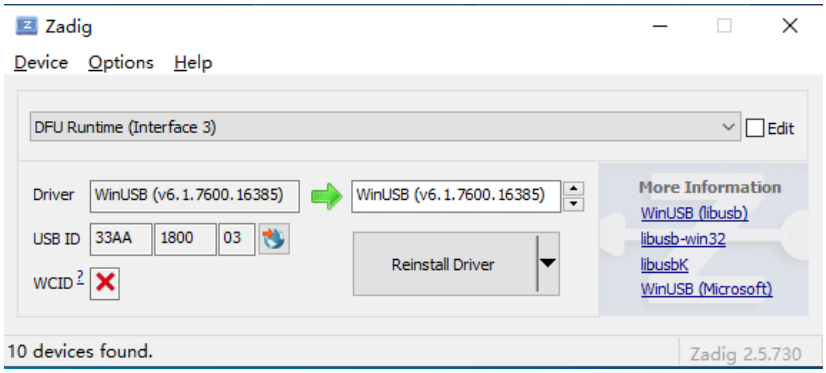
1. Open Gowin Programmer and configure the Access Mode to `External Flash Mode Arora V`.
2. In the advanced settings, manually change the Start Address to `0x300000`.
3. Select the pre-compiled Golden Image bitstream file (DFU file only) and execute the burn process.
4. In the advanced settings, manually change the Start Address back to `0x000000`.
5. Select the pre-compiled Working Image bitstream file and execute the burn process.

5.3 Environment Configuration

> ✅ The DFU upgrade function currently supports driverless operation. Once the device is connected, the Windows system will automatically load the driver—no extra configuration is needed (Plug and Play).

Note: If you are using an early firmware version (V1.0) or encounter automatic recognition issues, refer to the manual driver configuration steps below:

1. **Connect Device**: Connect the FPGA development board to the PC via a USB data cable and run Zadig as an administrator.
2. **Enumerate Devices**: In the top menu bar, click `Options` → `List All Devices` to ensure the tool reads all current physical USB devices.
3. **Select Target Hardware**: From the dropdown list on the main interface, select your DFU device (typically identified as **DFU Runtime (interface 0/3)**). Please double-check the device name to avoid misoperation.
4. **Assign Driver Protocol**: In the Target driver selection box on the right, toggle the arrows to select **WinUSB (v6.x.xxxx)**.



5.4 Using dfu-util for Upgrade Testing

This section primarily verifies the functionality of writing new firmware to the development board's external SPI Flash via the DFU interface. Two testing methods are provided: using the included GUI software or using the open-source command-line tool `dfu-util`.

5.4.1 Method 1: Upgrade via GUI Tool

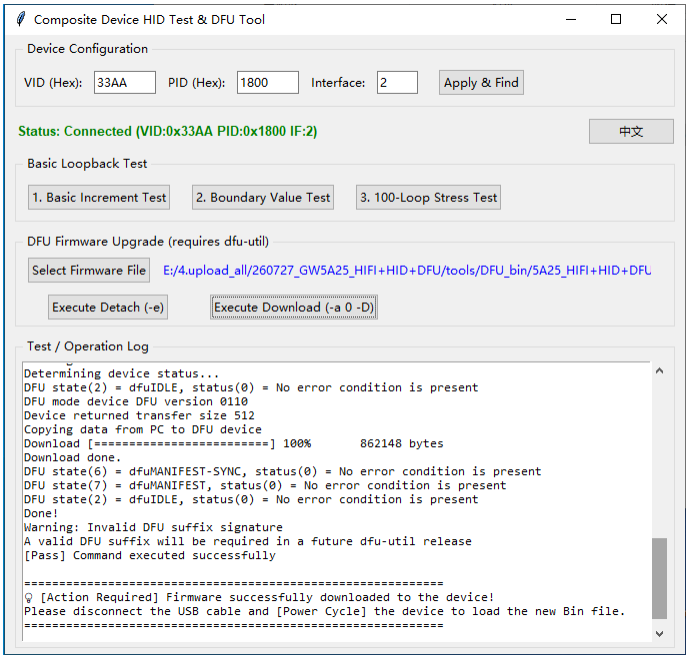
This test uses the **CDC+HID+DFU测试软件.exe** (included in the `tools` folder) to write new firmware to the board.

Test Environment & Requirements

- **Dependencies**: Ensure the test software is located in the same folder directory as `dfu-util.exe` and `libusb-1.0.dll`, or that the `dfu-util` path has been successfully added to the system environment variables. Otherwise, the download function will fail to execute.
- **Connection Status**: When performing the DFU upgrade test, the VID/PID and the current "Device Connection Status" on the software interface can be ignored.

Upgrade Procedure

1. **Select Firmware**: Click the "**Select Firmware File**" button and choose the desired `.bin` firmware file from the pop-up window.
2. **Enter DFU Mode**: Click the "**Execute Detach**" button to trigger the device to switch into DFU mode.
3. **Download Firmware**: Click the "**Execute Download**" button. The software will begin downloading the `.bin` file into the board's `spi_flash`. The output window below will display real-time download progress.
4. **Reboot to Apply**: Once the download reaches 100%, **power-cycle** the development board (disconnect and reconnect). The device will then load and run the newly burned firmware.



Test Firmware Description

The included `DFU_bin` folder provides several firmware files for testing:

- `5A60_USB2_HIFI+HID+DFU.bin` — A USB 2.0 composite device firmware designed for the DK_USB_GW5AT60-LV60UG225_V1.0 board, featuring HIFI+HID+DFU functionalities.
- `5A60_USB3_BULK+DFU_MSOS2.0_V1.1.bin` — A USB 3.2 composite device firmware for the DK_USB_GW5AT60-LV60UG225_V1.0 board, containing Bulk Bandwidth testing and DFU features.

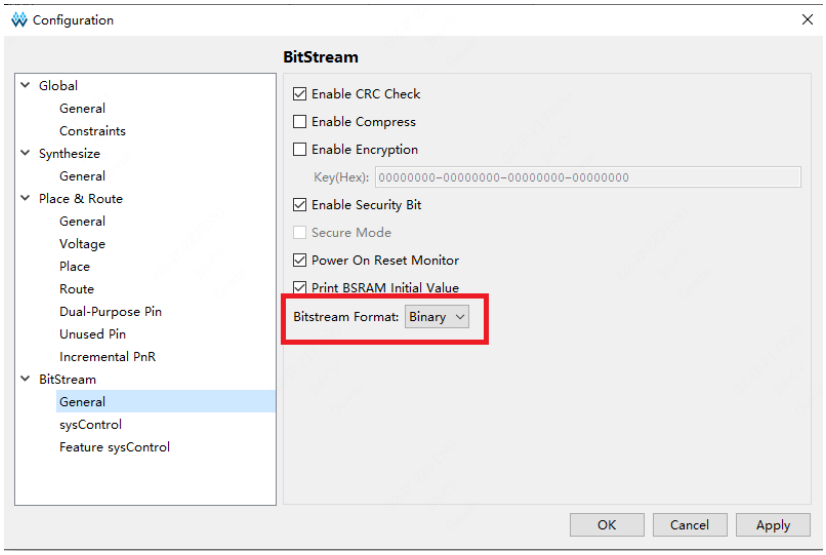
> ⚠ **Important Notice:**
> All the above firmware files retain the DFU upgrade capability. This means after burning these files and power-cycling, you can continue using the software for future firmware upgrades.

5.4.2 Method 2: Upgrade via dfu-util Command Line

You can also use the cross-platform open-source command-line tool `dfu-util` for testing. Ensure `dfu-util` has been added to your system environment variables.

Step 1: Prepare the Firmware Bitstream File

In the project synthesis tool, right-click and navigate to Configuration —— BitStream —— General —— Bitstream Format, and select **Binary**. After compilation, the software will generate corresponding `.fs`, `.bin`, and `.binx` firmware files in the `pnr` folder. Prepare the target `.bin` file to be updated (e.g., named `update_v2.bin`).



Step 2: Enumerate and Identify the Device

Open a command-line terminal (CMD / PowerShell) and enter the following command to check if the device is correctly identified:

```
dfu-util -l
```

If the driver is working correctly, the terminal will list all current DFU-supported devices, showing their corresponding VID and PID (e.g., `33AA:1800`).

Step 3: Deploy New Firmware

Execute the following command in the terminal to trigger the device to switch from APP mode to DFU download mode:

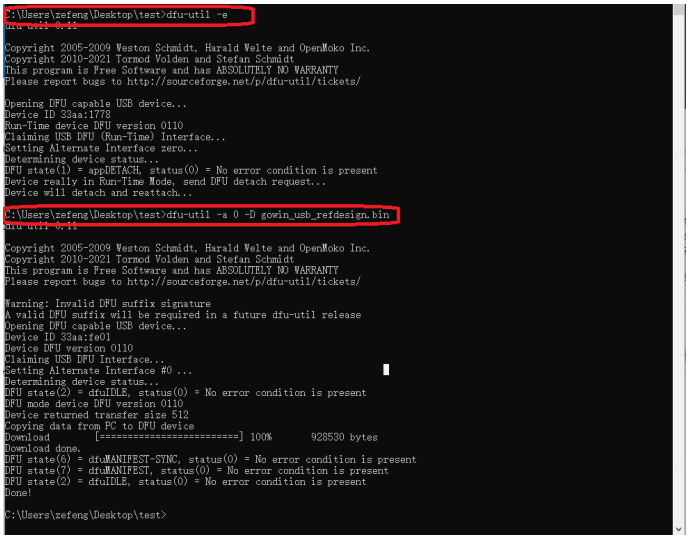
```
dfu-util -e
```

Next, initiate the download using this command:

```
dfu-util -d 33AA:1800 -a 0 -D update_v2.bin
```

Parameter Breakdown:

- `-d 33AA:1800`: Specifies the target device's VID:PID.
- `-a 0`: Targets Alt Setting 0 for the update, which the underlying logic automatically maps to Flash address `0x000000`.
- `-D`: Download command (downloads the specified firmware to the device).



Step 4: Background Upgrade Monitoring & Status

Upon executing the download command, a progress bar will appear in the terminal. The USB controller receives the data stream and drives the SPI Flash controller to perform erase/write operations on the `0x000000` sector.

> 💡 **Composite Function Concurrency Note:**
> When the device enters DFU mode, the OS will recognize it as a pure DFU device, temporarily disabling its original working functions. Once the DFU download finishes (progress reaches 100% and displays "Done"), the device will automatically revert to APP mode, allowing working functionalities to be tested again. However, note that the device is still running the old logic at this point; you must wait until the next power cycle for the FPGA to truly load the new firmware from `0x000000`.

Step 5: Reset & Multi-Boot Verification (Anti-Brick Testing)

1. **Normal Boot Verification**: Confirm the download is complete, disconnect power, and reconnect it. Observe whether the system operates normally according to the new firmware logic.
2. **Anti-Brick Fallback Verification (Destructive Test)**: You can attempt to download a corrupted `.bin` bitstream or one with a mismatched AES key to simulate an extreme firmware failure. After repowering, the Arora-V chip will fail the verification at `0x000000`, automatically triggering the Multi-Boot mechanism. It will fall back and load the Golden Image from the `0x300000` backup address. Once the system successfully boots via the Golden Image, you can use DFU again to overwrite the `0x000000` address with the correct bitstream, verifying the device's self-recovery capability.

```eof I have successfully translated your document to English while preserving all formatting, HTML structure, specific names, and corrections established previously! Let me know if you need anything else.
```